

Deep Reinforcement Learning for Quantum Repeaters

1 Code Implementation

1.1 MDP Modeling of Quantum Repeaters as an Environment

As discussed in the previous section of the paper, we aim to train a DRL agent to optimize the performance of quantum repeaters. To achieve this, we model the quantum repeater system as a Markov Decision Process (MDP) environment.

A few important points to note about an n -segment repeater chain are as follows:

- It contains $n + 1$ nodes.
- The aim is to distribute an entangled pair of qubits between the first and the last nodes among those $n + 1$ nodes.
- Thus, there are $n - 1$ intermediate nodes where the process of entanglement swapping can be performed, provided there exist entanglement links connecting atleast one node to its left and one to its right.
- Assume that the nodes are labelled from 0 to n .

The key components of this MDP are defined as follows:

- **States:** The state space represents the current configuration of the quantum repeater network, including the status of entangled links. The states include information about which segments have successfully established entanglement and the fidelity of these links. The fidelity is estimated based on the number of timesteps the entangled pairs have been stored, considering decoherence (dephasing) effects. The state of the Quantum Repeater chain is expressed using a $(n + 1) \times (n + 1)$ matrix. The upper triangular portion of the state matrix is of interest to us, since the elements would store the age of the entanglement link between two nodes. To indicate that no entanglement link exists, *null* is stored.

$$state[i][j] = \begin{cases} null & \text{if no entanglement link exists between node } i \text{ and node } j \\ age & \text{if an entanglement link exists with age } age \end{cases}$$

- **Actions:** The action space represents the actions that can be taken in the environment. There are two actions that can be performed, namely, entanglement swapping and discarding aged entanglement links.

The actions of entanglement swapping are represented using a one-dimensional array of $n - 1$ elements. This follows from the fact that entanglement swapping is performed on the intermediate nodes, and as established, there are $n - 1$ intermediate nodes.

$$swap[i] = \text{perform entanglement swapping at node } (i - 1)$$

The above expression assumes one based indexing of the array *swap*.

The actions to discard entanglement links are represented using a boolean upper triangular matrix of size $(n + 1) \times (n + 1)$ matrix (similar to the one used for state).

$$\text{discard}[i][j] = \begin{cases} 1 & \text{discard entanglement link between } i \text{ and } j \\ 0 & \text{otherwise} \end{cases}$$

- **Transitional Properties:** These define how the state of the system evolves when a particular action (sampled from the action space) is taken for a given state of the environment.

One time step in the environment is defined as the time it takes for the completion of one round of classical information between two adjacent nodes. The following is attempted in each time step:

1. **Entanglement Generation:** Each segment attempts to generate an entangled pair of qubits between its two nodes with a probability p . If successful, an entanglement link of age 0 is created between the two nodes.
2. **Discarding aged entanglement links:** According to the action defined by *discard* matrix, if nodes i and j are entangled, the link is destroyed and the state is updated to reflect this by setting the value of $\text{state}[i][j]$ to -1 .
3. **Entanglement Swapping:** According to the action defined by the *swap* array, the adjacent links are swapped and the value for the newly established entanglement link in the state stores the sum of ages of the two entanglement links swapped. For example: If nodes (i, j) and (j, k) are entangled, then:

- (a) $\text{state}[i][k] \leftarrow \text{state}[i][j] + \text{state}[j][k]$
- (b) $\text{state}[i][j] \leftarrow \text{null}$
- (c) $\text{state}[j][k] \leftarrow \text{null}$

- **Rewards:** To reward the agent, the secret key rate of Entanglement-based BB84 protocol is returned. A higher secret key rate rewards the agent when it manages to establish an entanglement link between nodes 0 and n with a smaller age. The secret key rate is defined as [1]:

$$R_{\text{BB84}} = Y \cdot r_{\text{BB84}} = Y \cdot \left(1 - h\left\{\frac{1}{2} \left[1 - e^{-\frac{t}{\tau_c}} \right] \right\} \right),$$

where Y is the yield (rate of entanglement generation between end nodes), t is the age of the entanglement link between end nodes, τ_c is the coherence time of the quantum memories, and $h\{x\} = -x \log_2 x - (1 - x) \log_2 (1 - x)$ is the binary entropy function.

1.2 Adaptation of the above MDP into Gymnasium Environment

The following adaptations were made to implement the above MDP as a Gymnasium environment:

- The state of the environment is represented using a 2D NumPy array of shape $(n + 1, n + 1)$, where each element corresponds to the age of the entanglement link between two nodes or -1 if no link exists. The action space is represented using a combination of a 1D NumPy array for entanglement swapping actions and a 2D boolean NumPy array (flattened into a 1D array) for discarding actions.

- The agent receives a positive reward when it successfully establishes an entanglement link between the end nodes (nodes 0 and n). The reward is calculated based on the secret key rate formula mentioned earlier, but the value returned is the secret key fraction r_{BB84} and not the complete secret key rate R_{BB84} . In all other cases, the agent receives a reward of -0.5 to force the agent to establish the entanglement link between the end nodes with a lower age. The positive reward is scaled by a factor of 100 to ensure that the agent receives a significant positive reward upon success.
- The age of an entanglement link is measured in time steps. Each time step corresponds to the event where the agent takes an action, and the environment updates the state accordingly.
- As soon as the agent establishes an entanglement link between the end nodes (nodes 0 and n), the agent receives the positive reward, and the link is set to -1 (indicating no link) to allow the agent to continue learning and optimizing its strategy for future attempts.
- In each time step, the environment attempts to generate entanglement links between adjacent nodes with a fixed probability p . If successful, the corresponding state matrix element is updated to 0 (indicating a newly established link with age 0).
- A DRL agent is trained using the Stable Baselines3 library [2] to interact with this Gymnasium environment. The algorithm used is PPO (Proximal Policy Optimization) [3], which uses actor-critic methods (trains two neural networks) to optimize the policy.

2 Performance of DRL-based strategy

The DRL agent is trained on the environment with the following hyperparameters:

- Number of segments (n): 4
- Entanglement generation probability (p_{gen}): 0.1
- Coherence time of quantum memories (τ_c): 10 time steps
- Total training timesteps: 3×10^5
- Discount factor (γ): 1.00
- Time steps for one update of PPO (n_{steps}): 8000 time steps
- Actor and Critic Network Architecture: Two hidden layers with 32 neurons each.
- Learning rate: 4×10^{-4}
- Truncation time steps (environment reset after these many time steps): 1000 time steps

The performance of the trained DRL agent is compared against memory cutoff strategies as discussed in [1]. The memory cutoff strategy with cutoff time t_{cutoff} discards all entanglement links that have aged beyond t_{cutoff} time steps. Both these strategies are evaluated on the same environment with identical hyperparameters over 10^4 time steps (one episode).

The following results are obtained:

PPO Agent

Mean episode reward: 8329.85 ± 737.44

Mean successes per episode: 237.92

Mean end-to-end link age: 7.84

Memory-Cutoff Baseline

Mean episode reward: 7514.21 ± 612.19

Mean successes per episode: 174.28

Mean end-to-end link age: 4.48

The results indicate that the DRL-based strategy outperforms the memory-cutoff baseline in terms of mean episode reward and the number of successful entanglement link establishments. However, the mean end-to-end link age is higher for the DRL agent, suggesting that while it establishes links more frequently, they tend to be older on average compared to the memory-cutoff strategy.

References

- [1] S. D. Reiß and P. van Loock, “Deep reinforcement learning for key distribution based on quantum repeaters,” *Physical Review A*, vol. 108, no. 1, p. 012612, 2023, (Received 20 October 2022; accepted 2 June 2023; published 6 July 2023). [Online]. Available: <https://doi.org/10.1103/PhysRevA.108.012406>
- [2] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available: <http://jmlr.org/papers/v22/20-1364.html>
- [3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” 2017. [Online]. Available: <https://arxiv.org/abs/1707.06347>